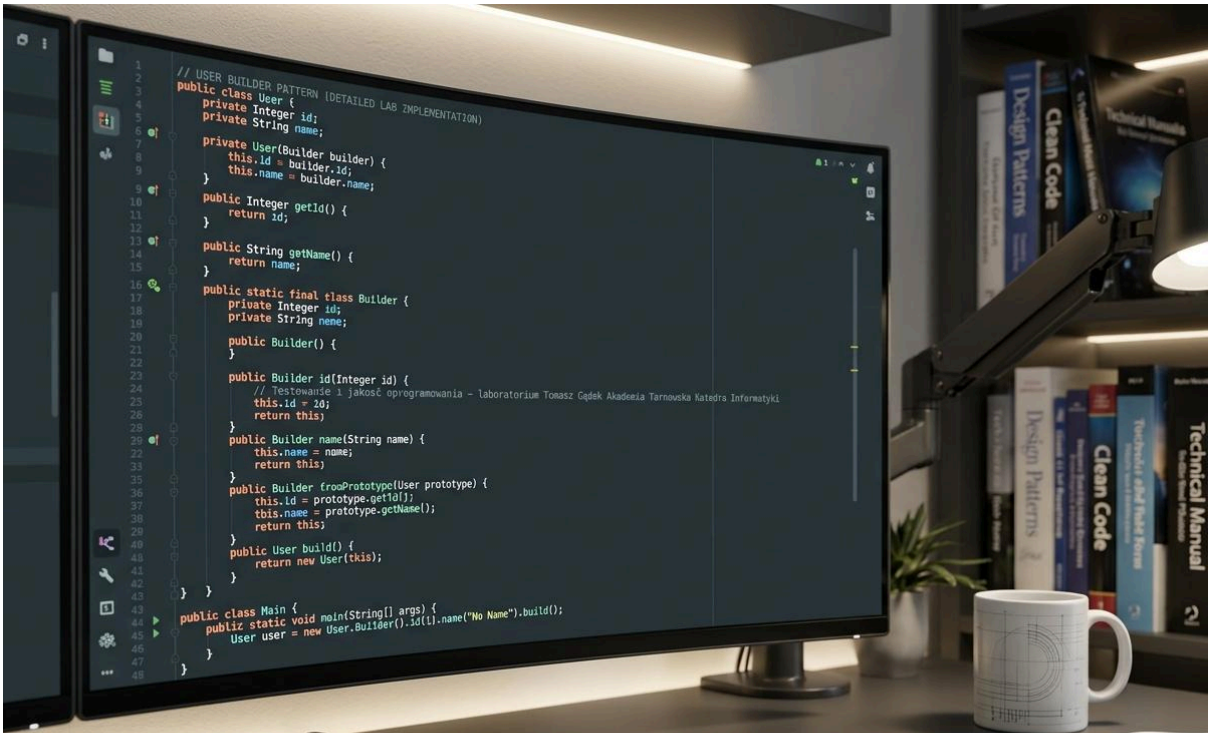
AKADEMIA TARNOWSKA

KATEDRA INFORMATYKI

Bezpieczeństwo i jakość oprogramowania

Laboratorium 3
DI, Builder, dokumentowanie API.



Tomasz Gądek

Cele laboratorium

Student potrafi:

- poprawnie wstrzykiwać zależności przy użyciu interfejsów.
- zaimplementować wzorzec Builder.
- wygenerować czytelną dokumentację REST API.

Zarys sytuacji

Klientowi podoba się obecna wersja systemu, ale przed zainwestowaniem w nowe funkcjonalności prosi o uporządkowanie istniejącego kodu. Twoje zadanie będzie polegało na refaktoryzacji projektu: musisz poprawić sposób, w jaki klasy współpracują ze sobą, ułatwić bezpieczne tworzenie obiektów oraz przygotować jasną dokumentację API.

Zadanie 1

Pobierz projekt **movies_part_03**:
https://drive.google.com/file/d/1SpoUenwIJ9tVtjYzYAbZQBe-XzyyL029/view?usp=sharing

Zadanie 2

Zrefaktoryzuj kod klasy **MovieServiceImpl**. Popraw wstrzykiwanie zależności tak, aby odbywało się ono wyłącznie poprzez interfejsy, a nie konkretne klasy. Na ten moment mechanizm wstrzykiwania zależności został zaimplementowany w sposób, który łamie zasady dobrego projektowania - **Dependency Inversion Principle** (5 zasada **SOLID**, czyli **DIP**).

Zadanie 3

Stosowanie setterów i konstruktorów o sporej liczbie parametrów przysparza wiele problemów. Możemy tego uniknąć stosując wzorzec projektowy **Builder**.

Przykładowa klasa zaimplementowana bez wzorca projektowego **Builder**:

```
class User {
    private Integer id;
    private String name;

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
```

```
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

class Main {
    public static void main(String[] args) {
        User user = new User();
        user.setId(1);
        user.setName("No Name");
    }
}
```

Implementacja przykładowej klasy zawierającej wzorzec projektowy **Builder**:

```
class User {
    private Integer id;
    private String name;

    private User(Builder builder) {
        id = builder.id;
        name = builder.name;
    }

    public Integer getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public static final class Builder {
        private Integer id;
        private String name;

        public Builder() {
        }
```

```
        public Builder id(Integer id) {
            this.id = id;
            return this;
        }

        public Builder name(String name) {
            this.name = name;
            return this;
        }

        public Builder fromPrototype(User prototype) {
            this.id = prototype.getId();
            this.name = prototype.getName();
            return this;
        }

        public User build() {
            return new User(this);
        }
    }
}

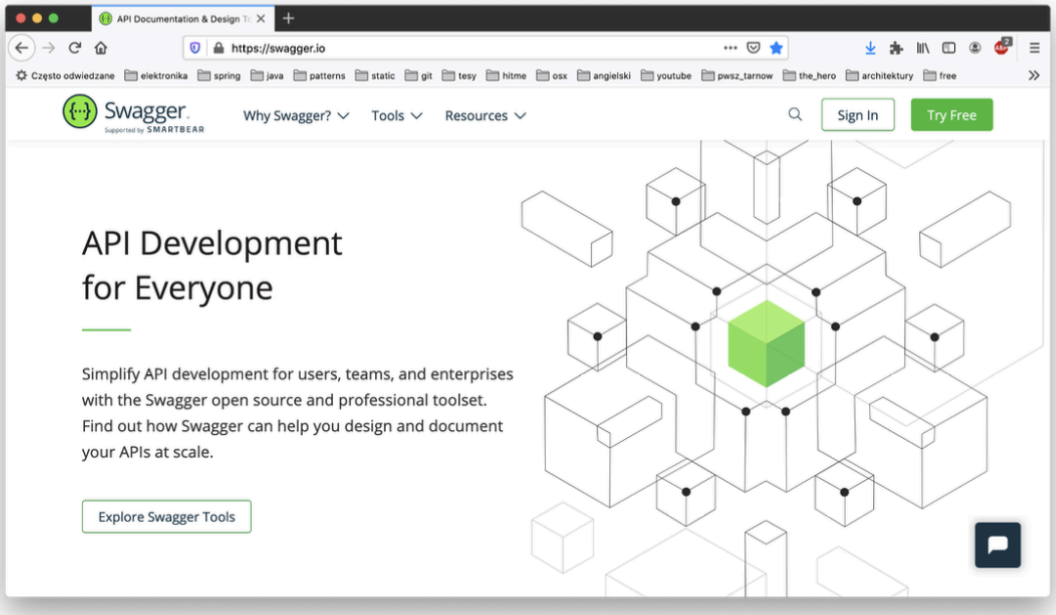
class Main {
    public static void main(String[] args) {
        User user = new User.Builder().id(1).name("No Name").build();
    }
}
```

Proszę zmienić implementację w klasach typu **DTO**, **ENCJA**. Proszę pozbyć się setterów i zastosować powyższy wzorzec.

Zadanie 4

Stwórz dokumentację techniczną Naszego API opierając się na usługach dostępnych w kontrolerze **MovieApiController**.

Załóż konto w usłudze https://swagger.io. Utwórz nowe API na podstawie szablonu **petstore**. Zmodyfikuj szablon w taki sposób, aby dokładnie odzwierciedlał usługi dostępne w Twojej aplikacji **movies**.


API Documentation & Design
https://swagger.io
Swagger
Why Swagger?
Tools
Resources
Sign In
Try Free
API Development
for Everyone
Simplify API development for users, teams, and enterprises with the Swagger open source and professional toolset. Find out how Swagger can help you design and document your APIs at scale.
Explore Swagger Tools